



3processes

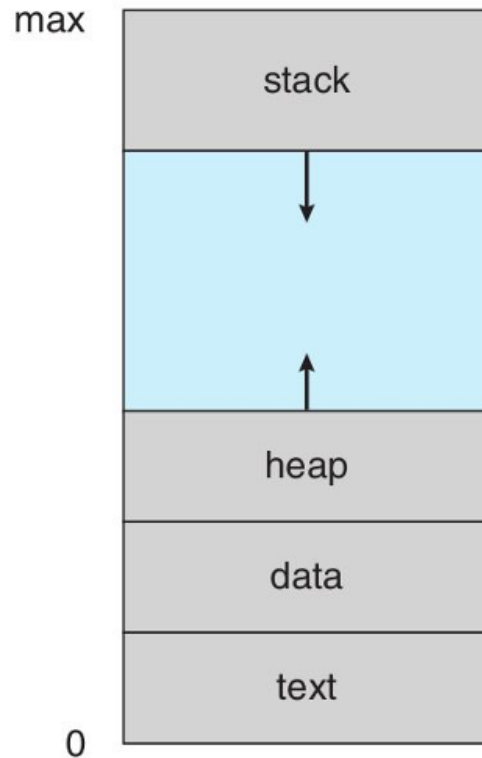
Process

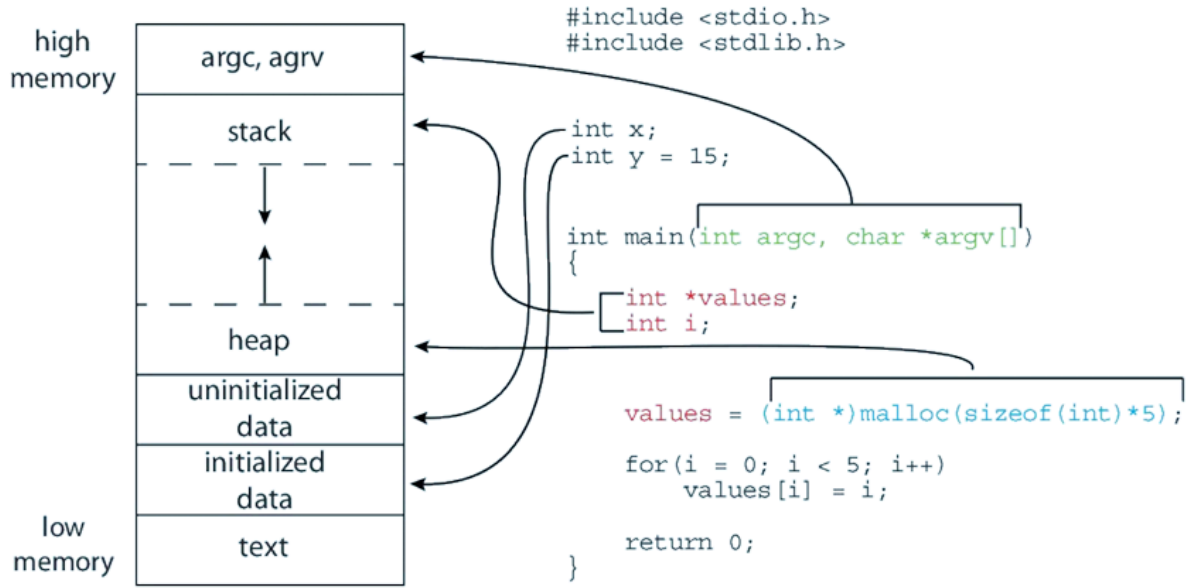
- Çalışan programlardır.
- Her süreç (process) sıralı şekilde teker teker çalışır, aynı process'in komutları paralel olarak aynı anda çalışmaz.
- Tek bir program sistemde birden fazla sürece dönüşebilir.

Bir Sürecin Bellek Düzeni

Bir süreç belleğe (RAM) yüklendiğinde, mantıksal olarak 4 ana bölüme ayrılır:

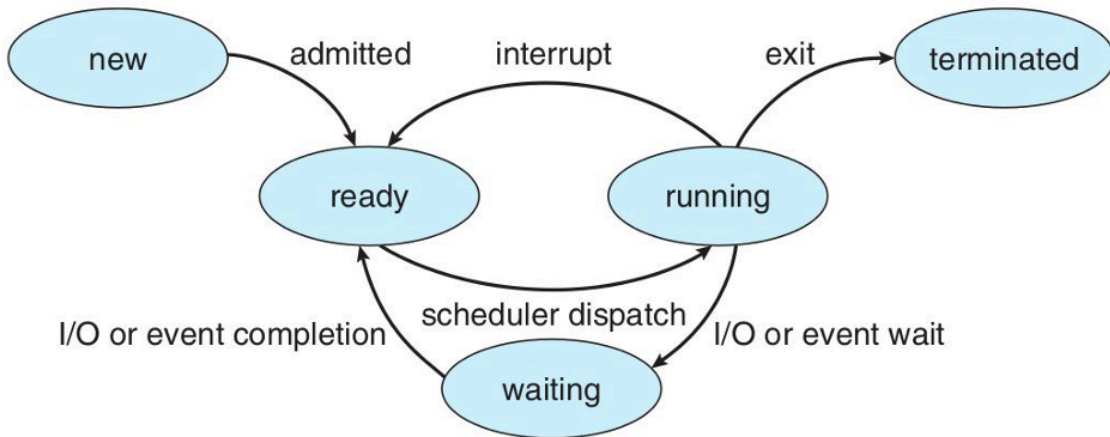
1. **Text (Metin/Kod) Bölümü:**
Programın derlenmiş komutlarının (kodlarının) tutulduğu kısımdır.
2. **Data (Veri) Bölümü:** Programdaki global (genel) değişkenlerin tutulduğu yerdir.
3. **Heap (Yığın/Öbek):** Program çalışırken (run time) dinamik olarak tahsis edilen bellektir (C dilindeki `malloc` ile ayrılan alanlar gibi).
4. **Stack (Yığıt):** Fonksiyon parametreleri, dönüş adresleri ve yerel (local) değişkenler gibi geçici verileri içeren kısımdır.





Process Durumları

- **New (Yeni):** Süreç henüz yaratılma aşamasındadır.
- **Ready (Hazır):** Süreç bellektedir ve işlemciye (CPU) atanmayı bekliyordur.
- **Running (Çalışıyor):** Sürecin komutları o an işlemci üzerinde aktif olarak çalıştırılıyordur.
- **Waiting (Bekliyor):** Süreç, bir Girdi/Çıktı (I/O) işleminin bitmesi gibi belirli bir olayın gerçekleşmesini bekliyordur.
- **Terminated (Sonlandı):** Süreç çalışmasını tamamen bitirmiştir.

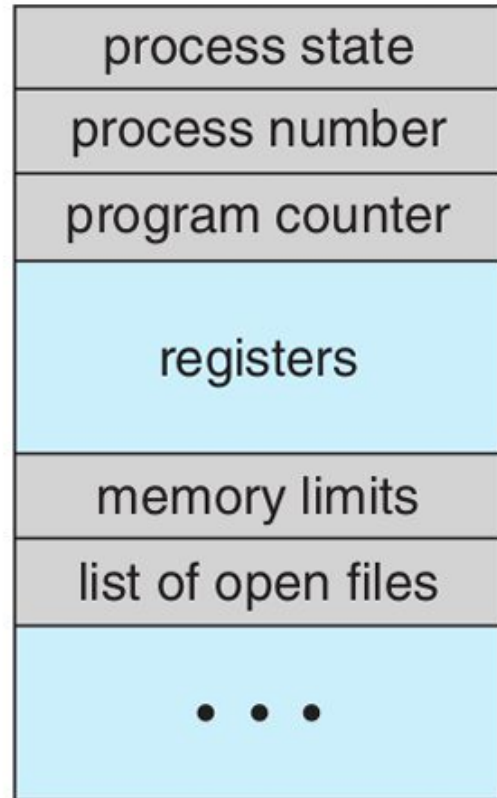


Süreç Kontrol Bloğu PCB

İşletim sistemi, sistemdeki her bir süreci takip edebilmek için onlara ait bilgileri **Süreç Kontrol Bloğu (Process Control Block)** adı verilen bir veri yapısında tutar.

PCB, sürecin kimlik kartı gibidir ve şu bilgileri içerir:

- **Process State (Süreç Durumu):** Sürecin o an hazırda mı, çalışıyor mu yoksa beklemede mi olduğu.
- **Process Number (Süreç Numarası - PID):** Sürecin sistemdeki benzersiz kimlik numarası.
- **Program Counter (Program Sayacı):** Sürecin çalıştıracağı bir sonraki komutun bellek adresi.
- **CPU Registers (İşlemci Yazmaçları):** Süreç durdurulduğunda CPU'daki yazmaçların o anki kopyası.
- **CPU Scheduling Info (Zamanlama Bilgisi):** Sürecin



öncelik seviyesi ve zamanlama kuyruklarındaki işaretçileri.

- **Memory-Management Info (Bellek Bilgisi):** Sürece tahsis edilen bellek sınırları.
- **Accounting Info (Muhasebe Bilgisi):** Sürecin ne kadar CPU ve saat süresi kullandığı.
- **I/O Status Info (I/O Durum Bilgisi):** Sürece atanmış cihazlar ve sürecin açtığı dosyaların listesi.

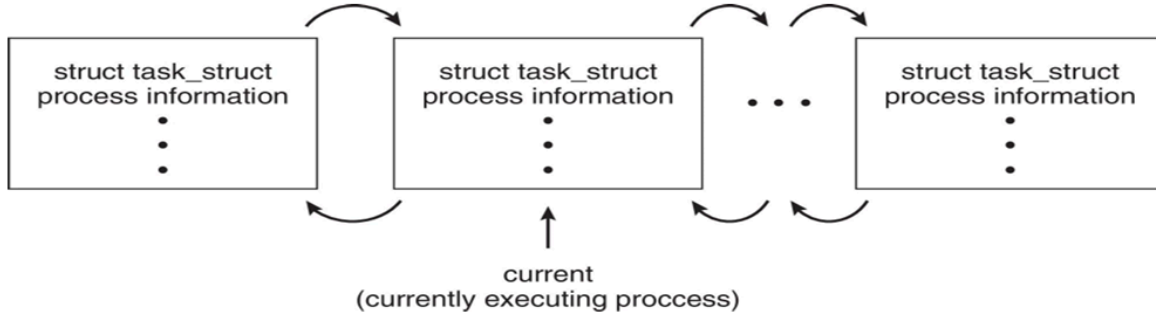
PCB Yapısı

- Linux işletim sistemi PCB'yi arkaplanda devasa bir C yapısı olan `data_struct` ile temsil eder.
- İşletim sistemi süreçleri (threads) birbirinden kopuk halde bırakmaz; onları çift yönlü bir bağlantılı liste şeklinde birbirine bağlar.
- Ayrıca işletim sisteminin elinde her zaman o an aktif olarak çalışan süreci gösteren bir `current` işaretçisi bulunur.

Threads

Şu ana kadar process'leri tek bir işi sırayla yapan (single thread) yapılar olarak düşündük. Ancak modern programlar aynı anda birden fazla iş yapmak ister.

- Örneğin; bir web tarayıcısı, bir yandan internetten dosya indirirken diğer yandan ekrandaki animasyonu oynatmak isteyebilir.
- **Çözüm:** İşletim sistemi, tek bir sürecin içine birden fazla Program Sayacı ekler. Böylece tek bir süreç içinde birden fazla kod noktası aynı anda çalışabilir. Bu durumda sürecin PCB'si, içindeki her bir iş parçacığının detaylarını her birini ayrı ayrı saklayacak şekilde genişletilir.



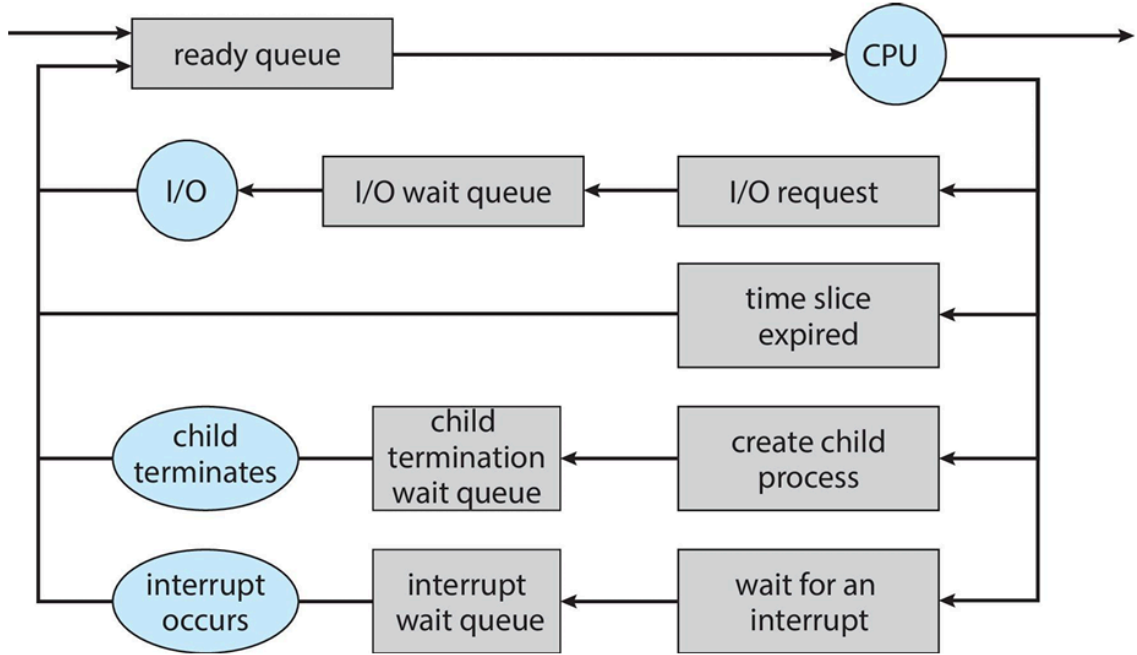
Process Scheduling

Sistemde aynı anda çalışmak isteyen onlarca thread varken, işlemciyi kimin, ne zaman kullanacağına Process Scheduler karar verir.

- **Maksimum verim sağlanır:** İşlemcinin kullanım oranını en üst düzeye çıkarmak (boş yatmasını engellemek).
- Süreçler arasında saniyenin binde biri hızında o kadar hızlı geçiş yapmak ki kullanıcı bilgisayarın başında otururken tüm programların aynı anda ve kesintisiz çalıştığını hissetsin.

Scheduling Queues

1. **Hazır Kuyruğu (Ready Queue):** Ana bellekte (RAM) bulunan ve tek eksiği işlemci (CPU) olan, çalışmaya tamamen hazır süreçlerin beklediği sıradır.
2. **Bekleme Kuyrukları (Wait Queues):** Çalışırken bir engelle takılan süreçlerin beklediği yerdir. Beklenen gerçekleştiğinde tekrar "Hazır Kuyruğuna" döner.



Process Operasyonları

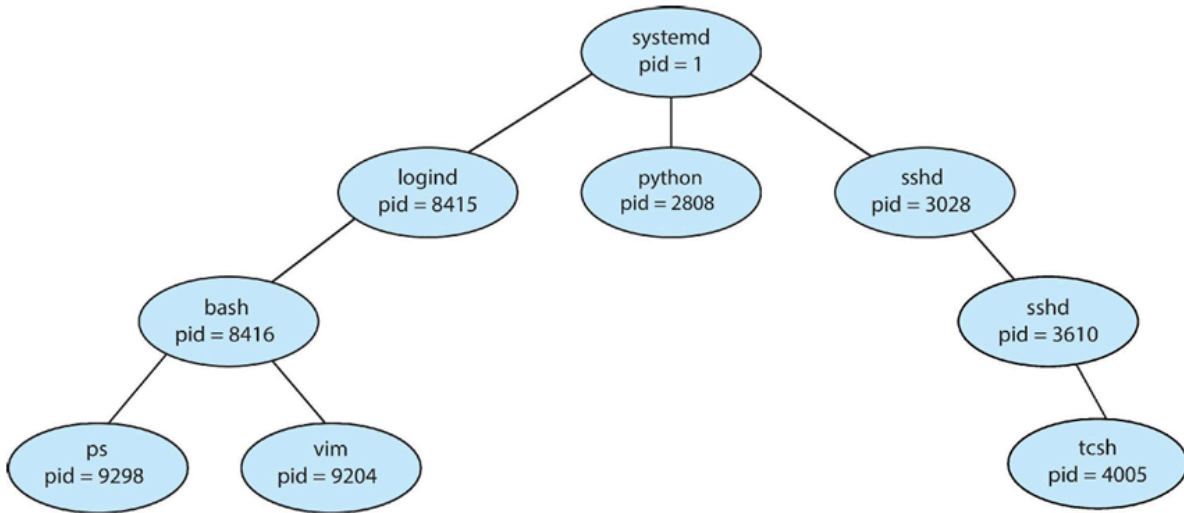
Her süreç mutlaka başka bir süreç tarafından yaratılır. Bu durum sistemde bir parent-child ilişkisi ve devasa bir soy ağacı oluşturur.

- **Kimlik Numarası (PID):** Sistemdeki her sürecin benzersiz bir süreç tanımlayıcısı (Process Identifier - PID) vardır.
- **Ataların Atası:** Linux/UNIX sistemlerinde bilgisayar açıldığında başlayan ilk süreç `systemd` (veya eski adıyla `init`) sürecidir ve PID numarası her zaman 1'dir. Diğer tüm programlar bu sürecin torunlarıdır.
- **Çalışma Seçenekleri:** Ebeveyn bir çocuk yarattığında, çocukla birlikte aynı anda çalışmaya devam edebilir (concurrent) veya çocuk işini bitirene kadar onu bekleyebilir (wait).
- **Parent-Child Arasında Farklı Kaynak Paylaşım Seçenekleri Vardır:**
 1. **Tam Paylaşım:** Ebeveyn ve çocuk sistemdeki tüm kaynakları tamamen ortak kullanır.
 2. **Kısmi Paylaşım (Alt Küme):** Çocuk süreç, ebeveynin sahip olduğu kaynakların sadece belirli bir alt kümesini (subset) kullanabilir. Ebeveyn çocuğuna bir sınır çizer.

3. **Sıfır Paylaşım:** Ebeveyn ve çocuk süreçler arasında hiçbir kaynak paylaşılmaz ``. İki süreç bellekte tamamen izole edilmiş yabancılar gibi takılır.

- **Çalıştırma/Yürütme Seçenekleri:**

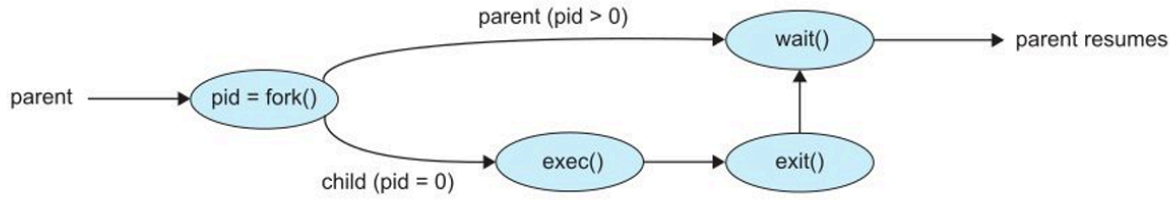
1. **Eşzamanlı Çalışma (Concurrent Execution):** Ebeveyn süreç kendi işine kaldığı yerden devam eder, çocuk süreç de aynı esnada kendi işini yapmaya başlar ``. İkisi sistemde yan yana, aynı anda aktiftir.
2. **Bekleme (Wait):** Ebeveyn süreç, çocuk süreç işini bitirip tamamen sonlanana (terminate) kadar kendi çalışmasını durdurur ve bekler.



Child Process Oluşturma

1. **fork()** - **Klonlanma:** Ebeveyn süreç (parent), **fork()** sistem çağrısını yapar. Bu komut, ebeveynin bellekteki birebir kopyasını oluşturur. Yeni doğan bu "çocuk sürecin" (child) tek farkı PID numarasının farklı olmasıdır.
2. **exec()** - **Dönüşüm:** Çocuk süreç klonlandıktan hemen sonra **exec()** çağrısı yapar. Bu çağrı, çocuğun belleğindeki o kopyalanmış eski kodları tamamen siler ve yerine çalıştırılmak istenen yepyeni programı yükler.
3. **wait()** - **Bekleyiş:** Ebeveyn süreç genellikle **wait()** çağrısı yaparak çocuğun işini bitirmesini bekler.

4. **exit()** - **Veda**: Çocuk işini bitirdiğinde **exit()** çağrısıyla sonlanır ve ebeveynine durumu hakkında bilgi göndererek (return status) sahneden çekilir.



Process Termination

- Bir süreç işini bitirdiğinde **exit()** çağrısıyla işletim sisteminden kendisini silmesini ve kaynaklarını (RAM vb.) serbest bırakmasını ister.
- Ancak bazen ebeveyn process, çocuğunu zorla öldürebilir (**kill**). Bunun üç temel sebebi vardır:
 - Çocuk süreç kendisine ayrılan kaynakları (bellek/CPU) aşmıştır.
 - Çocuğun yaptığı işe artık gerek kalmamıştır.
 - Cascading Termination (Zincirleme Katliam)**: Bazı işletim sistemleri kuralcıdır; eğer ebeveyn süreç ölürse, onun çocuklarının veya torunlarının yaşamasına izin vermez ve hepsini otomatik olarak sonlandırır.

Zombi ve Yetim Process'ler

- Zombi (Zombie) Process**: Çocuk süreç görevini başarıyla tamamlamış ve ölmüştür. Ancak, ebeveyni **wait()** çağrısı yapıp onun "ölüm raporunu/çıkış durumunu" teslim almamıştır. Çocuk fiziken ölüdür ancak işletim sisteminin PCB tablosunda bir kayıt olarak yer kaplamaya devam eder. Ruhunu teslim edememiş bir zombi gibidir.
- Yetim (Orphan) Process**: Bu senaryoda çocuk süreç hala tıkr tıkr çalışıyordur ancak ebeveyn süreç **wait()** komutuyla onu beklemeden aniden ölmüş veya sonlanmıştır. Sistemdeki bu yetim çocuğu genellikle ataların atası olan **systemd** (PID=1) evlat edinir ve onun arkasını toplar.

Mobil İşletim Sistemlerinde Hiyerarşiye Göre İşlem Sonlandırma

Masaüstü bilgisayarların aksine mobil cihazlarda bellek gibi kaynaklar çok daha kısıtlıdır. Sistem belleği dolduğunda, işletim sistemi bazı süreçleri zorla kapatarak kaynakları geri kazanmak zorundadır.

Android, süreçleri rastgele değil, kullanıcı deneyimini en az bozacak şekilde **en önemsizden en önemliye doğru** kapatmaya başlar

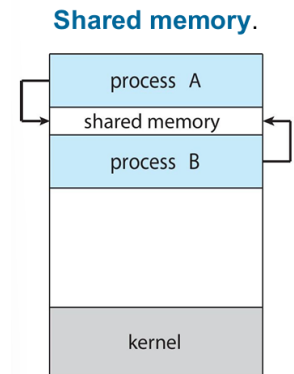
Hiyerarşi **en önemliden (kapatılması en zor/en son düşünülecek olandan) en önemsizye doğru** şu şekildedir:

1. **Ön Plan Süreci (Foreground process):** O an ekranda açık olan ve aktif olarak etkileşime girdiğin uygulamadır.
2. **Görünür Süreç (Visible process):** Doğrudan etkileşime girmediğin ama ekranda bir şekilde görünen veya ön planı etkileyen süreçlerdir.
3. **Servis Süreci (Service process):** Arka planda çalışan, aktif bir iş yapan işlemlerdir. (Spotify'dan müzik dinlemek veya bir dosya indirmek gibi).
4. **Arka Plan Süreci (Background process):** Açıp arka plana attığın, kullanıcının o an doğrudan görmediği ve aktif bir servis yürütmeyen uygulamalardır.
5. **Boş Süreç (Empty process):** İçinde hiçbir aktif uygulama kodu barındırmayan süreçlerdir. Sadece bir uygulama tekrar açıldığında daha hızlı başlatılabilirsin diye ön bellek (cache) mantığıyla tutulurlar. Bellek daraldığında ilk silinenler her zaman bunlardır.

Process'ler Arası İletişim (IPC)

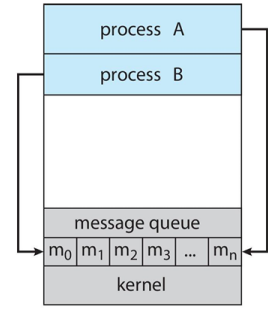
İşletim sistemlerinde süreçlerin haberleşmesi için iki büyük mimari vardır:

- **Paylaşımlı Bellek (Shared Memory):** İşletim sistemi iki sürecin de erişebileceği ortak bir RAM alanı ayırır.
 - *Artıları:* İnanılmaz hızlıdır ve büyük verileri paylaşmak için idealdir çünkü süreçler belleğe doğrudan erişir (overhead yoktur).
 - *Eksileri:* Güvenlik ve süreçlerin birbirini ezmemesi için senkronizasyon (synchronization) sağlamak tamamen programcının sorumluluğundadır.



- **Mesajlaşma (Message Passing):** Süreçler doğrudan birbirinin belleğine dokunamaz; kernel üzerinden birbirlerine mesaj (paket) atarlar.
 - *Artıları:* Çok daha güvenlidir, sistemler veya ağlar arası (uzaktan) haberleşmeye çok uygundur ve hata yönetimi kolaydır.
 - *Eksileri:* İşletim sistemi aracı olduğu için sürekli bağlam değişimi (context switch) gerektirir, bu da gecikme (latency) ve performans kaybı (overhead) yaratır.

Message passing.



Üretici - Tüketici Problemi (Shared Memory)

Bir süreç veri üretip ortak bir tampona (buffer) yazar, diğeri de oradan okur.

- Eğer tampon boyutu sınırsızsa (unbounded), üretici asla beklemek zorunda kalmaz.
- Tampon boyutu sınırlıysa (bounded), tampon dolduğunda üretici beklemek (wait) zorundadır.

Race Condition (Shared Memory)

- Üretici yeni veri eklediğinde `counter++` yapmak ister.
- Tüketici veri okuduğunda `counter--` yapmak ister.
- Eğer her iki süreç de **aynı anda** bu değişkene erişip değerini değiştirmeye kalkarsa, değişkenin son değeri hangi sürecin salise farkıyla işlemi son bitirdiğine bağlı olarak hatalı olur. Olayların zamanlamasına bağlı olarak sistemin hatalı/tutarsız davranmasına **Race Condition** denir.

Mesajlaşma Mekanizmaları (Message Passing)

İki process'in birbiriyle konuşabilmesi için iletişim bağlantısı kurulması şarttır.

İletişim Bağlantısı, sistemde 2 farklı düzlemde uygulanır:

1. **Fiziksel Olarak:** Paylaşımlı bellek üzerinden, donanımsal veri yoluyla (hardware bus) veya ağ (network) üzerinden yapılabilir.

2. **Mantıksal Olarak:** Doğrudan (Direct) veya Dolaylı (Indirect) olabilir. Senkron veya Asenkron çalışabilir.

Mantıksal Düzlemde İletişim Bağlantısı Çeşitleri

1. Direkt İletişim

- a. Süreçler birbirinin ismini tamamen bilmek zorundadır.
- b. **Özellikleri:** Bağlantı, süreçler iletişim kurmak istediği an otomatik olarak yaratılır. İki süreç arasında sadece tek bir bağlantı kurulabilir ve bu genellikle çift yönlüdür.
- **Simetrik İsimlendirme:** Her ikisi de birbirinin adını söyler. Gönderici `send(P, mesaj)` der, alıcı da `receive(Q, mesaj)` diyerek sadece Q'dan gelenleri kabul eder.

○ `send (P, message)`

■ send a message to process P

○ `receive(Q, message)`

■ receive a message from process Q

- **Asimetrik İsimlendirme:** Gönderici `send(Q, mesaj)` diyerek kime yolladığını söyler ama alıcı `receive(&mesaj)` diyerek kimden geldiğine bakmaksızın mesajı kabul eder (kimden geldiği mesajın içindeki kimlikte yazar).

2. Dolaylı İletişim

- a. Burada süreçler birbirlerinin adını bilmezler. İletişim, adına "**Posta Kutusu**" (**Mailbox / Port**) denilen benzersiz kimlikli araçlar üzerinden yapılır.

b. Süreçler mesajı doğrudan birbirine değil, kutuya atar: `send(Kutu_A, mesaj)` ve `receive(Kutu_A, mesaj)`). İki sürecin konuşabilmesi için aynı posta kutusunu paylaşıyor olmaları gerekir.

c. İki süreç arasında birden fazla bağlantı (birden fazla kutu) olabilir.

Eğer bir mailbox'ı birden fazla process dinliyorsa mesajı hangisi okumalı?

- Bir posta kutusuna aynı anda en fazla 2 sürecin bağlanmasına izin verilir.
- Kutudan aynı anda sadece tek bir sürecin `receive` (okuma) işlemi yapmasına izin verilir.
- İşletim sistemi mesajı alacak süreci rastgele (arbitrary) seçer ve kime gittiğini göndericiye bildirir.

Senkronizasyon

Blocking (Senkron / Beklemeli):

- *Blocking Send*: Gönderici mesajı yollar ve karşı taraf veya posta kutusu mesajı teslim alana kadar olduğu yerde donup bekler.
- *Blocking Receive*: Alıcı, kutuya geçerli bir mesaj gelene kadar boş boş bekler.

Rendezvous (Buluşma): Eğer hem gönderici hem de alıcı "Blocking" (beklemeli) modda çalışıyorsa, ikisi de mesajlaşma anında sistemde aynı anda durup buluşmuş olurlar. Buna Rendezvous denir.

Non-blocking (Asenkron / Beklemesiz):

- *Non-blocking Send*: Gönderici mesajı işletim sistemine teslim eder ve yerine ulaşmış ulaşmadığını umursamadan kendi işini yapmaya devam eder.
- *Non-blocking Receive*: Alıcı kutuya bakar. Mesaj varsa alır; yoksa beklemek yerine boş bir değer döndürüp yoluna devam eder.

Tamponlama

Mesajlaşma anında iletilen paketler havada duramaz; iletişim bağlantısının içine yerleştirilmiş geçici bir "mesaj kuyruğunda" (queue) tutulurlar.

1. **Sıfır Kapasite (Zero Capacity)**: Kuyrukta hiç yer yoktur. Bu yüzden gönderici, alıcı gelip o mesajı hemen elden teslim alana kadar beklemek zorundadır (Yani

Rendezvous zorunludur).

2. **Sınırlı Kapasite (Bounded Capacity):** Kuyruğun belirli bir sınırı (örneğin 10 mesaj) vardır . Eğer kuyruk dolmamışsa gönderici mesajı atıp yoluna devam edebilir. Ama kuyruk tamamen doluysa, yeni bir yer açılana kadar beklemek (block) zorundadır.
3. **Sınırsız Kapasite (Unbounded Capacity):** Kuyruk uzay boşluğu gibidir, asla dolmaz. Bu nedenle gönderici asla ve asla beklemes (never waits), sürekli yollamaya devam edebilir.

Pipes

Pipe Çeşitleri

- **Ordinary Pipes:** Klasik üretici-tüketici modelinde çalışır. Üretici borunun bir ucundan (write-end) yazar, tüketici diğer ucundan (read-end) okur .
 - Kesinlikle **tek yönlüdür (unidirectional)**.
 - Süreçler arasında mutlaka bir **ebeveyn-çocuk (parent-child) ilişkisi olmak zorundadır**. Genelde ebeveyn süreci boruyu yaratır ve kendi doğurduğu çocuk süreçle haberleşmek için kullanır.
- **İsimli Borular (Named Pipes):** Sıradan borulardan çok daha güçlüdür.
 - İletişim **çift yönlüdür (bidirectional)**.
 - İletişim kuracak süreçler arasında hiçbir akrabalık/ebeveyn-çocuk ilişkisi olmasına gerek yoktur.
 - Sistemdeki birden fazla farklı süreç bu boruyu haberleşmek için kullanabilir.

İstemci - Sunucu İletişimi ve Soketler (Sockets)

Süreçler her zaman aynı bilgisayarda olmak zorunda değildir. Bunun için iki temel yapı kullanılır: Soketler ve RPC.

Soket

İletişimin uç noktasıdır (endpoint). Bir soket, IP Adresi ve Port Numarasının birleşmesiyle oluşur.

- Örneğin; `161.25.19.8:1625` soketi, 161.25.19.8 IP adresindeki makinenin 1625 numaralı portunu işaret eder. Ağdaki iletişim her zaman bu şekilde bir çift soket arasında gerçekleşir .

1024'ün altındaki port numaraları (Örn: Web için 80) standart servisler için ayrılmış, "iyi bilinen (well known)" portlardır.

Eğer ağa çıkmadan, kendi bilgisayarındaki iki süreci soketle konuşturmak istersen `127.0.0.1` (**loopback**) IP adresini kullanırsın.

Remote Procedure Calls (RPC)

Ağ üzerindeki iletişimde soketler çok "düşük seviyeli" kalır.

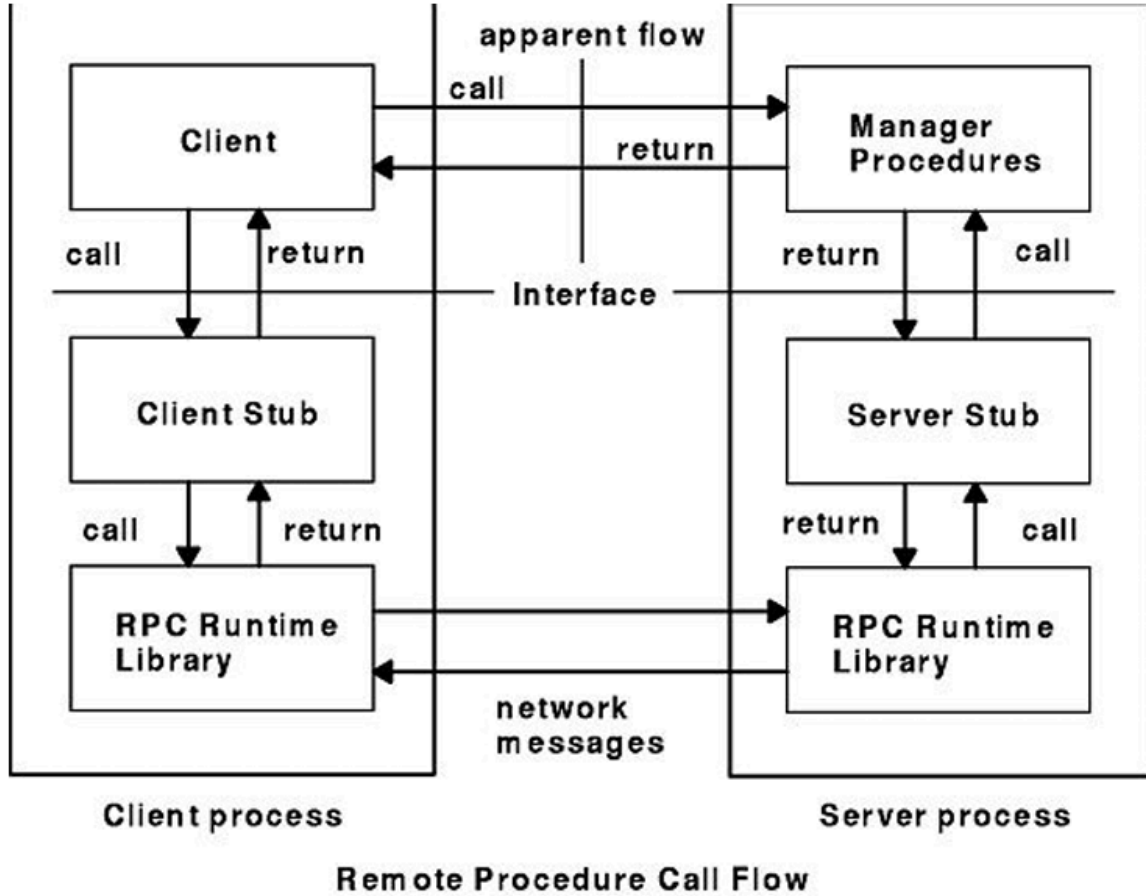
RPC (Remote Procedure Call), ağ üzerindeki başka bir bilgisayarda (sunucuda) bulunan bir fonksiyonu, sanki **kendi bilgisayarındaki normal bir fonksiyonmuş gibi (local procedure call)** çağırmanı sağlayan harika bir soyutlamadır.

RPC Akışı

- **Stub (Vekil/Taslak):** İstemci süreç (client), sunucudaki o fonksiyonun gerçek koduna sahip değildir. Bunun yerine kendi bilgisayarında o fonksiyonu taklit eden bir **İstemci Stub'ı (Client Stub)** çağırır. Bu çağrı tıpkı normal bir yerel fonksiyon çağırısı gibidir ve parametreler yığına (stack) eklenir.
- **Marshalling (Paketleme):** İstemci stub'ı, gönderilen parametreleri alır, ağ üzerinden gönderilebilecek bir mesaj formatına dönüştürüp paketler. Bu paketleme işlemine **Marshalling** denir.
- **Marshalling (Paketleme):** İstemci stub'ı, gönderilen parametreleri alır, ağ üzerinden gönderilebilecek bir mesaj formatına dönüştürüp paketler. Bu paketleme işlemine **Marshalling** denir.
- **İletişim:** İstemcinin işletim sistemi, bu mesajı ağ üzerinden sunucu makineye gönderir.
- **Unmarshalling (Paket Açma):** Sunucunun işletim sistemi mesajı alır ve **Sunucu Stub'ına (Server Stub)** iletir. Sunucu stub'ı bu mesajı açar, içindeki

parametreleri çıkarır (Buna ****Unmarshalling**** denir) .

- **Gerçek Çağrı:** Sunucu stub'ı, çıkardığı bu parametrelerle sunucudaki gerçek fonksiyonu/prosedürü çalıştırır. İşlem bitince sonuç aynı yolları tersten izleyerek istemciye geri döner .



RPC Zorlukları

- **Veri Temsili (Data Representation):** Senin bilgisayarın verileri "Little-endian" formatında (sondan başa) okuyor olabilir, sunucu ise "Big-endian" formatında (baştan sona) okuyor olabilir. Parametreler yanlış anlaşılabilir! Bunun çözümü için RPC, verileri ağa çıkmadan önce XDR (External Data Representation) adı verilen evrensel bir formata çevirir .
- **Hata İhtimali:** Yerel bir fonksiyon ya çalışır ya da sistem çöker. Ancak ağ üzerindeki bir çağrıda kablo kopabilir, paket kaybolabilir. Bu yüzden RPC

mesajları, normal sistemlere göre "en fazla bir kere çalıştır" (at most once) yerine "kesinlikle bir kere çalıştır" (exactly once) mantığıyla daha sıkı denetlenir.

- **Çöpçatan (Matchmaker):** İstemci, sunucudaki o fonksiyonun hangi portta çalıştığını nasıl bilecek? İşletim sistemleri, istemciyi doğru sunucu portuyla buluşturmak için "Rendezvous (Buluşma)" veya "Matchmaker" adı verilen bir rehberlik servisi sunar ``